Getting started with Dockerfiles might not be that difficult at the beginning, but building production-grade images can be challenging. In this section, you'll see how to improve the image you built in the previous section.

You'll use the layered-JAR feature provided by Spring Boot to build more efficient images. Then you'll consider essential security aspects related to container images. Finally, I'll discuss some factors to consider when choosing between Dockerfiles and Cloud Native Buildpacks for containerizing applications.

PERFORMANCE

When building container images, you should consider performance at build time and at run time. The layered architecture characterizing OCI images enables the caching and reusing of unchanged layers when building an image. Container registries store images by layers, so that when you pull a new version, only the changed layers are downloaded. That is quite an advantage in a cloud environment, considering the time and bandwidth you'll save for all your application instances.

In the previous section, you copied the Catalog Service standalone JAR file into a layer in the image. As a result, whenever you change something in your application, the whole layer must be rebuilt. Consider the scenario where you just add a new REST endpoint to your application. Even if all the Spring libraries and dependencies are unchanged, and the only difference is in your own code, you must rebuild the whole layer, since everything is together. We can do better. And Spring Boot can help us.

Putting uber-JARs inside a container image has never been efficient. A JAR artifact is a compressed archive containing all the dependencies, classes, and resources used by the application. All those files are organized in folders and